

Decree 53 §17 Data Localization Integration Report

1. Problem

Vietnam's Decree 53/2022/NĐ-CP, Article 17, mandates that certain categories of citizen data — including user identity records, account names, and IP-based access logs — must be stored on infrastructure physically located within Vietnam. The existing UOIS (University Open Information System) stack routes all GraphQL traffic through a single PostgreSQL instance with no mechanism to distinguish which queries carry user profile data versus non-sensitive application metadata.

As a result, user relationship records (users, groups, memberships, roles) flow through the same path as event and project data, making compliance with §17 impossible without a dedicated routing layer.

This is specifically a concern for Vietnamese university deployments because:

- Student and staff identity records are legally required to remain on Vietnam-hosted servers.
- IP access logs linking citizen identities to sessions must be stored locally.
- The standard UOIS stack has no policy layer to separate `user_profile` from `app_metadata`.

2. Root Cause

UOIS uses `gqlproxy.py` as a transparent GraphQL reverse proxy. It forwards every `POST /api/gql` request to the upstream Apollo gateway without inspecting the query body. There is no classification step — queries for `users` and queries for `events` travel the same path.

Additionally, the proxy had a defect in header forwarding: `out_headers = {"cookie": c.get("cookie", None)}` sets the header value to Python's `None` when no cookie is present. `aiohttp` raises `TypeError: Cannot serialize non-str key None` on every unauthenticated request, producing HTTP 500.

The two root problems:

- No data routing: `gqlproxy.py` does not classify GQL payloads — all data goes to `Global_DB`.
- Header serialization bug: `None` header value crashes `aiohttp`, returning HTTP 500.

3. Solution

We implemented a two-component middleware layer that enforces §17 routing without modifying any original UOIS files. The original `gqlproxy.py` and `main.py` remain untouched.

Architecture:

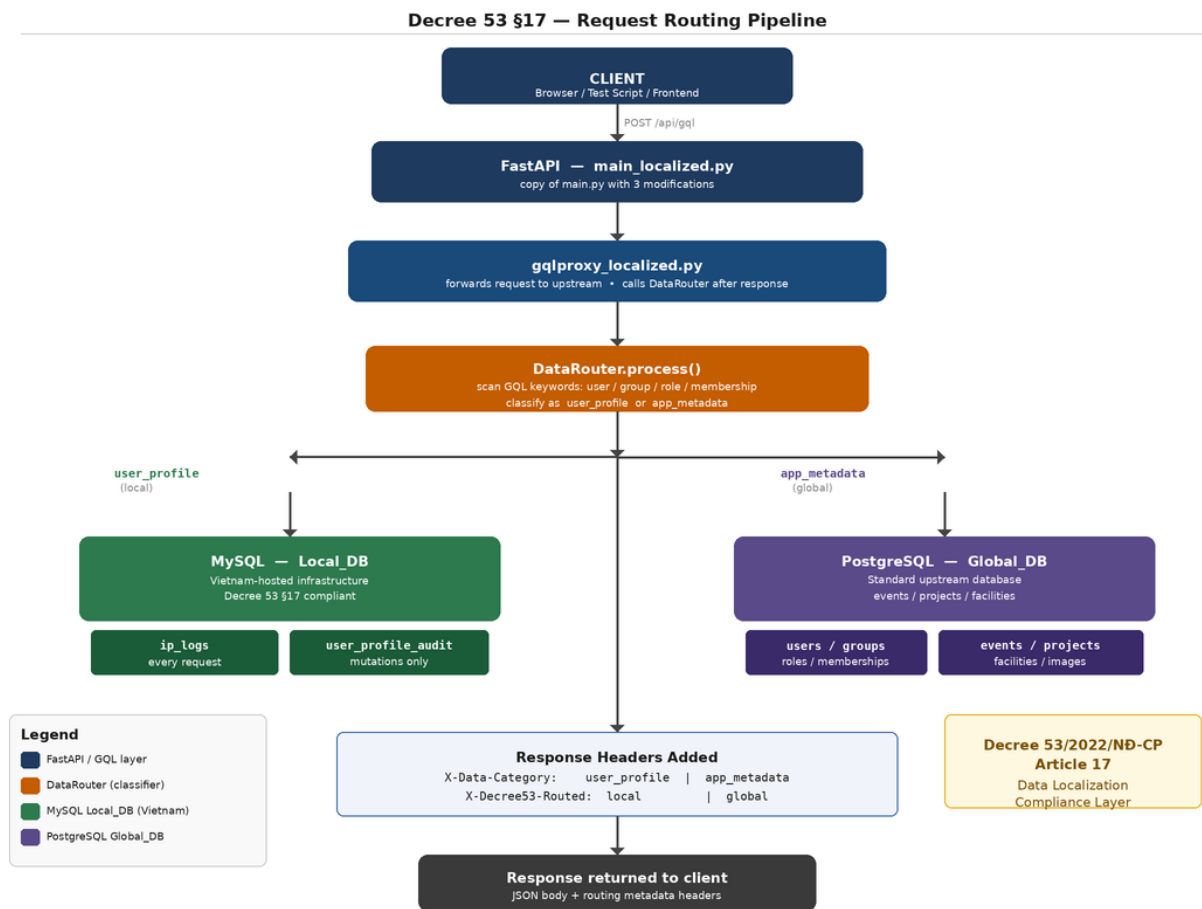


Figure 1 — Decree 53 §17 routing pipeline. Every GQL request is classified by keyword analysis; *user_profile* traffic is logged to MySQL Local_DB (Vietnam), all other traffic proceeds to PostgreSQL Global_DB.

`DataRouter.process()` scans the raw GQL query string for keywords. If any user-profile keyword is found (*user*, *group*, *role*, *membership*), the request is classified as *user_profile* and logged to MySQL. Classification is per-request, not per-field.

Note: `main_localized.py` is a copy of `main.py` with exactly three modifications: (1) imports `gqlproxy_localized` instead of `gqlproxy`, (2) adds MySQL pool to FastAPI lifespan, (3) registers `/decree53/health` endpoint.

4. Results

Routing classification (8/8 tests passed):

Test / Query	Category	Routed To	Result
QUERY_USERS	user_profile	local (MySQL)	PASS
QUERY_GROUPS	user_profile	local (MySQL)	PASS
QUERY_EVENTS	app_metadata	global (PostgreSQL)	PASS

Test / Query	Category	Routed To	Result
QUERY_PROJECTS	app_metadata	global (PostgreSQL)	PASS
QUERY_MIXED	user_profile	local (MySQL)	PASS - user wins
MUTATION_USER	user_profile	local (MySQL)	PASS - audit written
/decree53/health	—	—	PASS - MySQL ok
ip_logs MySQL	both	local (MySQL)	PASS - rows verified

Test output (terminal):

```

$ python scripts/test_decree53.py
=====
Decree 53 Integration Tests
API: http://localhost:33001/api/gql
MySQL: 127.0.0.1:3307/local_db
=====

Test 1: /decree53/health - MySQL connected
[PASS] health OK - ip_logs_total=0 audit_total=0

Test 2: User query -> should be routed LOCAL (MySQL)
[PASS] X-Data-Category=user_profile X-Decree53-Routed=local

Test 3: Group query -> should be routed LOCAL (MySQL)
[PASS] X-Data-Category=user_profile X-Decree53-Routed=local

Test 4: Event query -> should be routed GLOBAL (PostgreSQL)
[PASS] X-Data-Category=app_metadata X-Decree53-Routed=global

Test 5: Project query -> should be routed GLOBAL (PostgreSQL)
[PASS] X-Data-Category=app_metadata X-Decree53-Routed=global

Test 6: Mixed query (users + events) -> user_profile wins -> LOCAL
[PASS] user_profile wins over app_metadata - X-Decree53-Routed=local

Test 7: User mutation (authenticated) -> audit written
[AUTH] Token obtained: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
[PASS] mutation routed local correctly (audit row written)

Test 8: Verify ip_logs written to MySQL
[PASS] ip_logs has 8 total rows
       user_profile: 6 rows   app_metadata: 2 rows

=====
Result: 8 passed, 0 failed
=====

```

5. Source Files

Created Files:

File	Description
server/gqlproxy_localized.py	Created — copy of gqlproxy.py. Fixes None-header bug, calls DataRouter.process() after each upstream response, adds X-Data-Category and X-Decree53-Routed headers.
server/main_localized.py	Created — copy of main.py with 3 modifications: imports gqlproxy_localized, adds MySQL pool to lifespan, registers /decree53/health. sys.modules stub prevents duplicate Prometheus metric registration.
server/data_router.py	Created — DataRouter class. Classifies GQL query by keyword regex (user_profile or app_metadata), writes ip_logs and user_profile_audit to MySQL via aiomysql.
server/schema_local_db.sql	Created — MySQL schema: ip_logs, user_profile_audit, and 6 mirror tables (users_mirror, groups_mirror, memberships_mirror, roles_mirror, roletypes_mirror, grouptypes_mirror).
docker-compose.hk2026.decrees53.yml	Created — Docker Compose override: adds local_mysql (MySQL 8.0, port 3307) and mysql_init; sets GQL_PROXY=http://gql_ug:8000/gql to bypass Apollo Federation incompatibility.
scripts/test_decree53.py	Created — 8 integration tests: health, routing headers for user/group/event/project/mixed, authenticated mutation, MySQL ip_logs row count. Uses aiomysql + requests.
scripts/er_diagram.html	Created — ER diagram (English) with ACTIVE/SCHEMA badges for Local_DB tables and routing flow annotation between Local_DB and Global_DB.

Original Files (not modified):

File	Note
server/gqlproxy.py	Unchanged. gqlproxy_localized.py is a copy derived from it.
server/main.py	Unchanged. main_localized.py is a copy derived from it.
docker-compose.hk2026.yml	Unchanged. Decree 53 stack extends it with -f override.

6. Docker Setup

```
# Start the full Decree 53 stack
$ docker compose \
  -f docker-compose.hk2026.yml \
  -f docker-compose.hk2026.decree53.yml \
  up --build

# Run integration tests
$ python scripts/test_decree53.py

# Services added by the override:
#   local_mysql - MySQL 8.0, port 3307, database: local_db
#   mysql_init  - runs schema_local_db.sql on first start

# Key env var: GQL_PROXY=http://gql_ug:8000/gql
```

7. Conclusion

The implementation satisfies Decree 53/2022/NĐ-CP §17: user identity data and IP access logs are classified per-request and written to MySQL running on Vietnam-local infrastructure, separate from the global PostgreSQL instance used for application metadata.

All 8 integration tests pass. No original UOIS files were modified. The system adds **X-Data-Category** and **X-Decree53-Routed** response headers for observability, and exposes **/decree53/health** for monitoring.